

4月13日报

本日学习内容

- 1. 自动布局子视图的基本了解(子视图的宽,高可变化, 距离父视图的上边缘, 下边缘, 左边缘, 右边缘的距离可变化)
- 2. UI中事件响应链的基本了解 (响应链传递顺序, 命中测试顺序)

今日算法题

题目1: [48. 旋转图像](#)

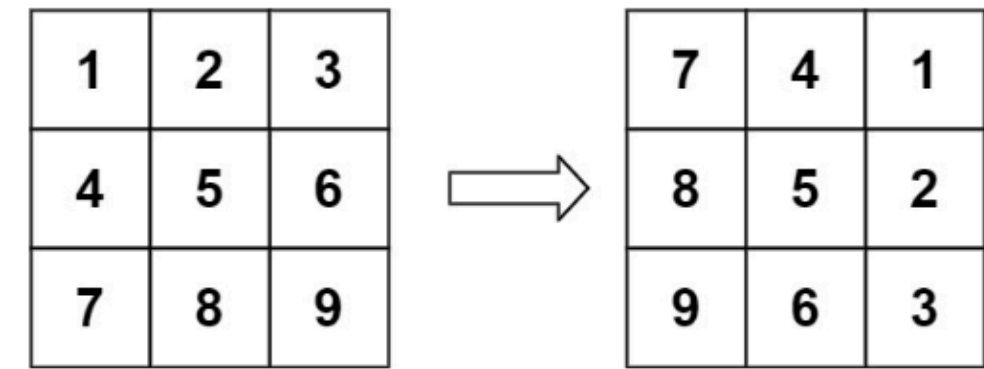
48. 旋转图像

已解答

中等 相关标签 相关企业 Aa

给定一个 $n \times n$ 的二维矩阵 `matrix` 表示一个图像。请你将图像顺时针旋转 90 度。
你必须在 [原地](#) 旋转图像，这意味着你需要直接修改输入的二维矩阵。请不要 使用另一个矩阵来旋转图像。

示例 1:



输入: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`
输出: `[[7,4,1],[8,5,2],[9,6,3]]`

```
1 // #define ll long long
2 // class Solution {
3 // public:
4 //     void rotate(vector<vector<int>>& matrix) {
5 //         ll n = matrix.size();
6 //         for (ll i = 0; i < n; i++) {
7 //             for (ll j = 0; j < i; j++) {
8 //                 swap(matrix[i][j], matrix[j][i]);
9 //             }
10 //         }
11 //         for (auto& row : matrix) {
```

```

12 //          reverse(row.begin(), row.end());
13 //      }
14 //  }
15 // };
16
17
18
19
20 // #define ll long long
21 // class Solution {
22 // public:
23 //     void rotate(vector<vector<int>>& matrix) {
24 //         ll n = matrix.size();
25 //         for (ll i = 0; i < n; i++) {
26 //             for (ll j = i + 1; j < n; j++) {
27 //                 swap(matrix[i][j], matrix[j][i]);
28 //             }
29 //             ranges::reverse(matrix[i]);
30 //         }
31 //     }
32 // };
33
34
35 #define ll long long
36 class Solution {
37 public:
38     void rotate(vector<vector<int>>& matrix) {
39         ll n = matrix.size();
40         for (ll i = 0; i < (n / 2); i++) {
41             for (ll j = 0; j < (n + 1) / 2; j++) {
42                 ll temp = matrix[i][j];
43                 matrix[i][j] = matrix[n - j - 1][i];
44                 matrix[n - j - 1][i] = matrix[n - i - 1][n - j - 1];
45                 matrix[n - i - 1][n - j - 1] = matrix[j][n - i - 1];
46                 matrix[j][n - i - 1] = temp;
47             }
48         }
49     }
50 };

```

本日遇到的问题

1. 事件响应链中命中测试顺序和响应链传递顺序记忆混乱
2. 自动布局子视图中学习太过浅,还需利用AI工具深入学习

明日学习计划

1. 跟随课程学习UI操作